

26 — ETL Progress Dashboard: Real-Time Pipeline Monitoring

If you haven't read doc 14 (pipeline orchestration) yet, that one covers how the ETL runs end-to-end — this doc assumes that background and focuses on a different problem: while the ETL is running (potentially for hours), how does an admin know what's happening?

What it is

The ETL progress dashboard is a three-layer system that lets an admin watch the pipeline's progress in real time through a web browser:

1. **Writer (ETL side):** The pipeline writes its current progress (phase, processed count, skipped count, failed count, total) to the `app_config` database table as a JSON value under the key `etl_progress`.
2. **API endpoint:** A FastAPI `GET /admin/etl/progress` endpoint reads that JSON from the database and returns it.
3. **Frontend (Admin UI):** A React component polls the endpoint every 5 seconds using `setInterval`, calculates a percentage, and renders a live progress bar.

No WebSockets, no message queues, no push notifications — just a database row, an HTTP endpoint, and a polling timer. Simple, reliable, and easy to debug.

Real-life analogy

Imagine a construction foreman building a housing development of 100 houses. Every time a house is completed (or skipped because the lot has a problem, or fails inspection), he walks to a whiteboard in the site

office and updates the tally: "42 built, 3 skipped, 1 failed, 54 remaining."

The project manager doesn't stand next to the foreman all day. Instead, she checks the whiteboard from her office window every 5 minutes. The whiteboard is always there, always shows the latest numbers, and doesn't require the foreman to stop working to give her an update.

- The **whiteboard** is the `app_config` row with key `etl_progress`.
- The **foreman updating it** is the `_update_etl_progress()` function called every 5 records.
- The **manager checking the window** is the React component's `setInterval` polling every 5 seconds.
- The **site office** is the PostgreSQL database — both the foreman (ETL) and the manager (API) can access it independently.

Why we used it here

The ETL pipeline processes thousands of records and each one involves multiple API calls with retry/backoff (docs 11, 25). A full run can take 30 minutes to several hours. Without progress visibility, an admin running the pipeline would see nothing but a silent terminal until it finishes (or fails) — no way to know if it's 10% done or 90% done, how many records are failing, or whether it's stuck.

We chose **database-as-message-bus** over more complex alternatives because:

- **The database is already there.** Both the ETL and the API server already connect to the same PostgreSQL instance. No new infrastructure (Redis, RabbitMQ, WebSocket server) to deploy or maintain.
- **The ETL and API are separate processes.** The ETL runs as a standalone script (or Docker container); the API server is a separate FastAPI process. They don't share memory. But they both talk to the same database — so writing progress to a DB row from the ETL and reading it from the API is the simplest possible inter-process communication.

- **Polling is fine at this scale.** One small JSON read every 5 seconds from a single row is negligible load. WebSockets would add complexity (connection management, reconnection logic, state synchronization) for no meaningful benefit when the data changes at most once per second.

Code walkthrough

Layer 1: The ETL writes progress

From `bilingual_etl/scripts/main_etl.py`:

```
def _update_etl_progress(conn, data: dict) -> None:
    try:
        with conn.cursor() as cur:
            cur.execute(
                "INSERT INTO app_config (key, value) VALUES ('etl_progress', %s::j"
                "ON CONFLICT (key) DO UPDATE SET value = EXCLUDED.value",
                (json.dumps(data),),
            )
            conn.commit()
    except Exception as e:
        logger.warning(f"Failed to update ETL progress: {e}")
```

This uses PostgreSQL's `INSERT ... ON CONFLICT DO UPDATE` (also called "upsert") — if the `etl_progress` key doesn't exist yet, it inserts it; if it does, it replaces the value. The entire progress state is stored as a single JSON blob, so updating it is one atomic SQL statement.

The `try/except` wrapping means a progress-write failure (unlikely but possible — e.g., a transient connection issue) doesn't crash the ETL. Progress reporting is informational; the actual data processing continues regardless.

Where it's called:

```
def process_categories(conn, categories, masker, force=False, limit=None, workers=
    total = len(categories)
    processed = skipped = failed = 0
```

```

def _report():
    _update_etl_progress(conn, {
        "phase": "categories",
        "total": total,
        "processed": processed,
        "skipped": skipped,
        "failed": failed,
        "status": "running",
        "started_at": int(time.time()),
    })

_report() # Initial report: "starting categories phase"

for category in categories:
    # ... process category, update processed/skipped/failed counts ...
    if (processed + skipped + failed) % 5 == 0:
        _report() # Report every 5 records

_report() # Final report for this phase

```

Reports happen at three points: start of a phase, every 5 records, and end of a phase. The "every 5" interval means the dashboard updates roughly every few seconds during active processing — frequent enough to feel live, infrequent enough to not waste time on DB writes.

Completion summary:

```

completion_summary = {
    "phase": "complete",
    "status": "complete",
    "categories": {"processed": cat_processed, "skipped": cat_skipped, "failed": c
    "companies": {"processed": comp_processed, "skipped": comp_skipped, "failed":
    "finished_at": int(time.time()),
}
_update_etl_progress(conn, completion_summary)

```

When the entire ETL finishes, the progress JSON changes shape — it now contains a `status: "complete"` and a breakdown per entity type (categories and companies). The frontend detects this status change

and switches from showing a progress bar to showing a completion summary.

Layer 2: The API endpoint

From `search_api/routers/admin_config.py`:

```
@etl_router.get("/admin/etl/progress")
def get_etl_progress():
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute("SELECT value FROM app_config WHERE key = 'etl_progress'")
            row = cur.fetchone()
    finally:
        conn.close()
    if not row:
        return {"status": "idle"}
    return row[0]
```

The simplest possible endpoint: read one row, return the JSON value. If no ETL has ever run (the row doesn't exist), return `{"status": "idle"}`. No authentication required on this endpoint — it's read-only progress data, not a config mutation.

Layer 3: The React frontend

From `admin-ui/src/pages/EtlProgress.jsx`:

Polling setup:

```
useEffect(() => {
    function poll() {
        getEtlProgress().then(setProgress).catch(() => {});
    }
    poll(); // Fetch immediately on mount
    timer.current = setInterval(poll, 5000); // Then every 5 seconds
    return () => clearInterval(timer.current); // Cleanup on unmount
}, []);
```

`useEffect` with an empty dependency array (`[]`) runs once when the component mounts. It calls `poll()` immediately (so you see data right away, not after a 5-second delay), then sets up a `setInterval` that calls `poll` every 5000 milliseconds (5 seconds). The return function cleans up the interval when the component unmounts — without this, the timer would keep firing after you navigate away from the page, causing unnecessary network requests and potential memory leaks.

Progress bar calculation:

```
const isRunning = progress.status === "running";
const done = (progress.processed || 0) + (progress.skipped || 0) + (progress.failed || 0);
const total = progress.total || 1;
const pct = Math.round((done / total) * 100);
```

`done` is the sum of all completed records (whether they succeeded, were skipped via hash gate, or failed). `total` defaults to 1 to avoid division by zero. The percentage drives a CSS `width` on the progress bar fill element.

Conditional rendering for three states:

1. **Idle** (`status === "idle"`): Shows a message "No ETL run recorded yet" with the command to start one.
2. **Running** (`status === "running"`): Shows stat cards (processed/skipped/failed counts) plus an animated progress bar with percentage.
3. **Complete** (`status === "complete"`): Shows a final summary grid with separate breakdowns for categories and companies, plus the completion timestamp.

The stat cards layout:

```
<div className="stat-grid">
  <div className="stat-card">
    <div className="stat-label">Processed</div>
    <div className="stat-value" style={{ color: "var(--success)" }}>
      {progress.processed || 0}
    </div>
  </div>
</div>
```

```
    { /* ... similar cards for Skipped and Failed ... */ }  
  </div>
```

The `stat-grid` and `stat-card` classes come from the Admin UI's shared CSS. Failed count turns red (`var(--danger)`) only when non-zero, so it visually draws attention only when there's a problem.

How to explain this in an interview

"We built real-time ETL progress monitoring using a simple three-layer architecture: the ETL pipeline writes its progress counts as a JSON blob to a PostgreSQL `app_config` row every 5 records, a FastAPI endpoint reads and returns that row, and a React component polls the endpoint every 5 seconds using `setInterval`. We chose this polling-over-database approach because the ETL and API are separate processes that already share a database — no new infrastructure needed. The frontend handles three states: idle (no run yet), running (progress bar with percentage), and complete (summary breakdown). The progress write is wrapped in a try/except so a failure never crashes the actual data pipeline — monitoring is informational, not on the critical path."